Mechanistic Interpretability of Antibody Language Models Using SAEs

Rebonto Haque[1] Oliver Turnbull[1] Anisha Parsan[2,3] Nithin Parsan[2] John J. Yang[2] Anna L. Beukenhorst[4,5]

[1]Department of Statistics, University of Oxford, UK [2]Reticular, San Francisco, USA [3]EECS, MIT, Cambridge MA, USA [4]Leyden Laboratories BV, Leiden, The Netherlands [5]Department of Epidemiology, Harvard T.H. Chan School of Public Health, Boston, MA, USA

Correspondence: deane@stats.ox.ac.uk

Abstract

Sparse autoencoders (SAEs) are a mechanistic interpretability technique that have been used to provide insight into learned concepts within large protein language models. Here, we employ TopK and Ordered SAEs to investigate autoregressive antibody language models, and steer their generation. We show that TopK SAEs can reveal biologically meaningful latent features, but high feature–concept correlation does not guarantee causal control over generation. In contrast, Ordered SAEs impose a hierarchical structure that reliably identifies steerable features, but at the expense of more complex and less interpretable activation patterns. These findings advance the mechanistic interpretability of domain-specific protein language models and suggest that, while TopK SAEs suffice for mapping latent features to concepts, Ordered SAEs are preferable when precise generative steering is required.

1. Introduction

Antibodies are a key part of the body's adaptive immune response. They are characterised by their ability to bind to a specific antigen and subsequently neutralise it or initiate an immune response. Their extensive sequence—and therefore structural—diversity enables binding to virtually any target antigen (Chiu et al., 2019).

The antigen-binding domain of antibodies is made up of variable heavy (VH) and variable light (VL) chains. Antibody-antigen binding specificity and affinity are largely determined by structural units known as complementarity-determining regions (CDRs), with each VH and VL chain having 3 CDRs, making a total of 6. Within our genome, there are numerous V, D, and J gene segments which together code for the VH and VL chains.

The combinatorial assembly of the discrete gene segments–V (variable), D (diversity), and J (joining) for the VH (heavy) chain, and V and J for the VL (light) chain–give rise to the final sequence diversity within the VH/VL chains. Somatic hypermutations, characterised by random nucleotide substitutions occurring at rates considerably higher than the genomic background, in the joint V(D)J segment further increases sequence diversity (Andreano & Rappuoli, 2021).

The ability to bind any target antigen with high specificity and affinity makes antibodies ideal candidates for drug discovery. As a result, antibody drugs hold a major and growing share of the total pharmaceutical market (Crescioli et al., 2025). Antibody drug development pipelines need to identify candidates which bind specifically and with high affinity to the target antigen, while also being 'developable' (Jarasch et al., 2015). 'Developability' refers to properties required for a successful drug such as immunogenicity, solubility, specificity, stability, manufacturability, and storability (Raybould & Deane, 2022).

Antibody language models have been used to optimise multiple steps of antibody-drug development pipelines from library generation (Turnbull et al., 2024; Shuai et al., 2023; Nijkamp et al., 2023) and light chain generation (Capel et al., 2026) to humanisation during lead optimisation (Chinery et al., 2024; Prihoda et al., 2022; Marks et al., 2021). In this work we will primarily use p-IgGen, a GPT-like decoder-only model trained on antibody-sequence data, consisting of 17M parameters (Turnbull et al., 2024). The authors released a paired model, as well as a finetuned version capable of generating diverse antibody libraries with developable properties.

The lack of interpretability of machine learning models contributes to a lack of trust in model predictions, difficulty determining whether biologically relevant features are being used to make predictions and difficulty detecting overfitting. Collectively, these pose a barrier when employing language models for drug discovery (Chen et al., 2023). SAEs offer a promising approach to identify human-interpretable concepts learned by models and steer their generation (Chen et al., 2025; Templeton et al., 2024). Prior works have used SAEs to understand the inner mechanisms of protein language models (PLMs) (Adams et al., 2025; Parsan et al., 2025b; Simon & Zou, 2024), and steer model output. However, to date, SAEs have not been used to interrogate autoregressive protein or antibody-specific language models.

In this work we aim to advance the interpretability of antibody language models, using SAEs to identify biologically relevant features of interest learned by p-IgGen, and predictably steer its generation. We identify antibody-specific features, such as the complementarity-determining region (CDR) identity and germline gene identity, and use them to steer p-IgGen generation for specific germline gene identities. While p-IgGen is our primary mechanistic case study, we additionally evaluate whether the interpretability–steerability patterns we observe are specific to a single architecture. To this end, we replicate key analyses on two additional antibody language models—IgLM (Shuai et al., 2023) and ProGen2-OAS (Nijkamp et al., 2023)—using the same SAE-based pipeline. These experiments are not intended as exhaustive model

comparisons, but as evidence that the qualitative conclusions we draw from p-IgGen generalize across scale and architecture.

Overall, we demonstrate the applicability of SAEs for incorporating rational design principles to antibody library generation. We show that TopK SAEs can accurately identify interpretable latents underpinning model generation, whereas Ordered SAEs can identify steerable features capable of tuning model generation.

2. Related Work

2.1. Mechanistic Interpretability

Mechanistic interpretability refers to the approach of explaining complex machine learning systems through the behaviour of their functional units (Kästner & Crook, 2024) by decomposing or reverse-engineering systems into their more elementary computations (Rai et al., 2025). The eventual goal is to discover causal relationships between model inputs and corresponding outputs.

Within the context of transformer-based language models, there are three main ideas relevant for mechanistic interpretability research: features, circuits and universality (Rai et al., 2025). Features refer to human-interpretable properties that are encoded by model activations (Templeton et al., 2024). Circuits inform how these features are extracted from model inputs and processed to influence model outputs (Olah et al., 2020b). Finally, universality determines whether features and circuits identified for a specific model and task exist in other models and tasks (Olah et al., 2020a). This paper specifically focuses on the identification of features from language models and using these features to steer model generation.

2.2. Sparse Autoencoders

Sparse Autoencoders (SAEs) have specifically been employed in mechanistic interpretability for feature discovery. They are able to tackle the issue of feature superposition resulting in polysemantic neurons, where any given neuron encodes multiple, often unrelated features. SAEs overcome this problem by projecting dense neuron activations into a sparser latent space using a sparse encoder, Equation 1, whilst ensuring the latent representation can be reconstructed back into the original neuron representation by a decoder following sparsification, Equation 2.

$$z = g(ReLU\big(W_{\text{enc}}\, x + b_{\text{enc}}\big)) \tag{1}$$

$$\hat{x} = W_{\text{dec}}\, z + b_{\text{dec}} \tag{2}$$

where W are the weight matrices and b are the bias vectors, enc and dec denote the encoder and decoder respectively,

x is the original hidden representation, z the latent representation, and $\hat{x}$ the reconstructed hidden representation. $ReLU$ activation is applied to the latent representation following encoding and g is a sparsification function.

2.2.1. TopK SAEs

TopK SAEs (Gao et al., 2024) limit the number of active latents to k, where $k \ll d_{\text{in}} \ll d_{\text{sae}}$. d_{in} is the input hidden dimensions, and d_{sae} is the latent or dictionary dimensions. Equation 3 shows the loss computation.

$$L(x) = \underbrace{\|x - \hat{x}\|_2^2}_{\text{Reconstruction loss}} + \underbrace{c}_{\text{Sparsity constraint}} \tag{3}$$

The $L(x)$ reconstruction loss compares the decoded representation $\hat{x}$ with the original hidden representation x. When a sparsification function is not directly applied during encoding, a separate sparsity constraint is added in loss computations, which is usually a variation of an L1 regularisation loss (Zhang et al., 2018).

2.2.2. Ordered SAEs (O-SAEs)

Ordered SAEs follow a nested SAE architecture, enabling hierarchical ordering of SAE latents. Importantly, compared to the traditional TopK SAE architecture which arbitrarily orders hierarchical latents within the dictionary space, O-SAEs enforce a strict, consistent, hierarchical ordering of latents. This is because TopK SAEs enforce sparsity within the entire dictionary space in one go, whereas O-SAEs follow a nested approach and effectively train a number of individual, nested SAEs which occupy an increasing portion of the dictionary space.

O-SAEs introduce two core components: (i) *per-index nested grouping*, and (ii) *strictly decreasing truncation weights* in order to ensure consistent ordering.

(i) For each truncation level $m \in \{1, \ldots, d_{\text{sae}}\}$, the first m rows of the encoder and decoder are isolated:

$$W_{\text{enc}}^{(m)} = [W_{\text{enc}}]_{1:m,:}, \quad W_{\text{dec}}^{(m)} = [W_{\text{dec}}]_{1:m,:} \tag{4}$$

In Eq. (4) the encoder–decoder pair $\big(W_{\text{enc}}^{(m)}, W_{\text{dec}}^{(m)}\big)$ re-uses the first m rows of the full weight matrices. Because every smaller autoencoder is a strict subset of the larger one, any latent $i \le m$ is shared across all groups that follow. This "per-index nested grouping" forces early latents to model global structure that remains useful for every deeper stage. Per-index grouping ensures non-random sampling of dictionary sizes, unlike in Matryoshka SAEs (Bussmann et al., 2025), increasing the overall consistency of results.

(ii) Each partial reconstruction is weighted by a *monotonically decreasing* probability $p_M(m)$, so that early (low-index) features incur a higher penalty when failing to capture coarse structure. The per-truncation loss is

$$L_m(x) = p_M(m)\, \Big\| x - W_{\text{dec}}^{(m)\top}\, W_{\text{enc}}^{(m)}\, x \Big\|_2^2 \tag{5}$$

and summing over all m promotes the model to learn the most "abstract" elements first, with progressively finer details later. Combining the decreasing probability weights with nested latents further enforces ordering of identified latents and maintains a stricter hierarchy.

3. Methods

3.1. Sparse Autoencoder Training

We adapted TopK Sparse Autoencoders (SAEs) from the EleutherAI/sparsify GitHub repository:

(https://github.com/EleutherAI/sparsify), and Ordered SAEs were implmented based on the paper by (Wang et al., 2025). Training parameters were taken directly from the original repositories where available.

3.1.1. Base models and activation extraction

Our primary experiments use p-IgGen, an autoregressive antibody language model trained on paired OAS. To test robustness, we also run the same SAE training and concept-probing pipeline on IgLM and ProGen2-OAS, which differ in architecture and scale. We trained both the TopK and Ordered SAEs on hidden layer activations of respective models, generated from the original p-IgGen training set (Turnbull et al., 2024). For p-IgGen, we extract activations from each of its 4 hidden layers. For IgLM and ProGen2-OAS we only extract activations from their final layers.

The training set contained **1,800,545** VH/VL paired sequences from the Observed Antibody Space database (OAS) (Olsen et al., 2022; Kovaltsuk et al., 2018). When generating p-IgGen activations for training, we concatenated the paired VH and VL sequences together, with appropriate start and end tokens added, and passed them into p-IgGen to generate hidden activations. This generated 4 sets of hidden activations, one from each hidden layer. For Ordered SAE training, we randomly subsampled **100,000** sequences to decrease training time.

We further trained SAEs on final hidden layer activations of IgLM and ProGen2-OAS using the same p-IgGen training sets as before. Since IgLM and ProGen2-OAS were originally trained using unpaired sequences, we generated activations for unpaired VH and VL chains with appropriate start, end, and padding tokens from the respective vocabularies.

3.1.2. TopK SAE

The model's input dimensions d_{in} were projected onto a higher-dimensional latent/dictionary size d_{sae}, where $d_{\text{sae}} = d_{\text{in}} \times r = d_{\text{in}} \times 32$. $r = 32$ is the expansion factor. ReLU activation was applied to the projection, $z = \text{ReLU}\big(W_{\text{enc}}\, x + b_{\text{enc}}\big)$, followed by a Top-$k$ sparsification with $k = 32$, retaining only the top 32 activations by magnitude. The resulting dictionary size was **32x**. Decoder weights were initialised as the unit-normalised transpose of the encoder weights to stabilise training. Training used a batch size of 8 and Adam optimisers throughout, with a custom learning rate $\eta = \frac{2\times10^{-4}}{\sqrt{d_{\text{sae}}/16{,}384}}$.

3.1.3. Ordered SAE

Ordered Sparse Autoencoders (O-SAEs) were adopted to retain higher-level, abstract features within our latent space and hierarchically arrange the latents. In our setup, we used expansion factor $r = 8$, yielding a dictionary size $d_{\text{sae}} = d_{\text{in}} \times r = d_{\text{in}} \times 8$. All models were trained with Adam optimisers at a fixed learning rate $\eta = 1 \times 10^{-4}$. We chose a smaller maximum dictionary size for the O-SAEs to speed up training, effectively reducing the total number of nested SAEs being trained. Due to per-index grouping, O-SAEs need to train several nested SAEs based on the total dictionary size, whereas the regular TopK architecture only trains a single model.

For p-IgGen, we initially used Batch Top-K activation for sparsification, with $k = 32$ (following an earlier version of (Wang et al., 2025)), where activations across all residues in a batch are pooled and the top $k \times B$ selected globally (B = batch size). This maintains k active latents on average, but allows per-residue sparsity to vary. Subsequently, IgLM and ProGen2-OAS used standard Top-K (fixed k per residue) for fairer comparison with Top-K SAEs.

3.2. Targeted Feature Identification using SAEs

3.2.1. Training data

Paired antibody sequence data were obtained from OAS, Coronavirus Antibody Database (CoV-AbDab) (Raybould et al., 2021), and the Patent and Literature Antibody Database (PLAbDab) (Abanades et al., 2024). A total of 149,069 sequences were obtained from the respective datasets, based on their binding specificities to SARS-CoV2 RBD (binder and non-binder).

The data was clustered based on CDR sequence similarity using CD-HIT (Li et al., 2001), with a 0.8 similarity threshold on the total CDR sequence. The clusters were then randomly split into the training-validation-test set, whilst ensuring members of the same cluster were in only one of the three possible splits. The splits were further stratified based on binding specificity to SARS-CoV2 RBD.

This specific dataset was originally prepared for a separate project, and the SARS CoV2 RBD binding properties of the antibodies are not relevant for this study. Qualitatively, a dataset of equivalent size randomly sampled from OAS should produce the same results.

The following concepts were studied to identify associated latents: CDR identity, which refers to whether a given residue lies within a specific CDR region, and V/J gene identity, which refers to the germline V or J gene segment was used to code for the final antibody sequence. For the CDR-identity, the training matrix was the latent activations for each residue. The CDR identity dataset had 7 classes (6 CDR identities and non-CDR regions). For sequence-level concepts, V/J gene identity, the training matrix was the mean pool of the *non-zero* residue-level latent activations in a given sequence.

3.2.2. Linear Probe

We trained a logistic regressor to act as a linear probe on the training-validation data. A logistic regressor (LR) was trained, employing 3-fold cross-validation grid search to optimise hyperparameter C. In logistic regression, C is the inverse of the regularisation strength: larger C applies less regularisation and can overfit, while smaller C applies more regularisation and can improve generalisation. Cross-validation was done during training by randomly shuffling and splitting the training data into 3 cross-validation sets. Correlation weights of all latents were stored and the latents with the top 500 positive correlation weights were used for further validation.

3.2.3. Latent Selection

The top correlated latents were further validated on the validation set. Based on the strategy by Simon and Zhou (Simon &